

Multiprocessing in Python

Parallelism

До этого мы жили с предположением, что у нас одно ядро CPU, поэтому в данный момент времени мог выполняться только один поток (процесс)

Идея: если есть несколько ядер, то можно запускать несколько потоков (процессов) одновременно

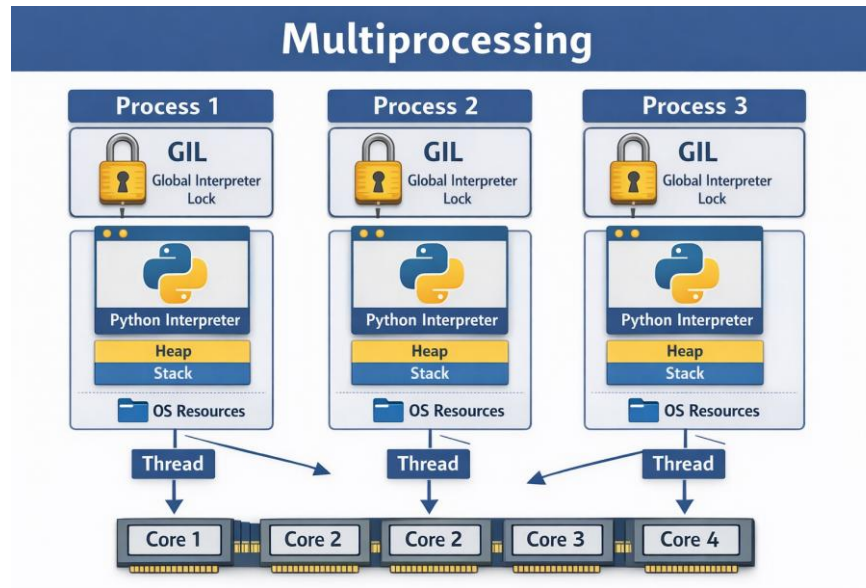
Parallelism. Multiprocessing

Multiprocessing — это модель параллелизма, при которой создаётся несколько процессов и они могут выполняться в один момент времени на разных ядрах CPU

Когда мы создаём новый процесс:

- свое адресное пространство
- свой process context в ОС (PID, потоки), при этом часть ресурсов **может наследоваться** (файлы/сокеты)
- как минимум один поток
- у каждого процесса свой экземпляр интерпретатора python
- у каждого процесса свой GIL

Поэтому *multiprocessing*, как правило, выгоден для “тяжелых” **CPU-bound** задач, потому что иначе будет неоправданным создание процессов и их взаимодействие (IPC)



Примечания

Несколько замечаний:

1. Потоки *могут быть запланированы* ОС на разные ядра, но в CPython это **не даёт параллельного выполнения Python-байткода** из-за GIL
2. Если происходит параллельное I/O в нескольких потоках из разных процессов, то сможет ли устройство обработать несколько запросов одновременно, зависит уже от самого устройства (например, HDD и SSD)
3. Несколько потоков могут одновременно делать CPU операции, если вычисления идут **в нативном коде, который освобождает GIL**. Например, **polars** на **Rust** считает параллельно, или использование в нескольких потоках кода с компилятором типа **numba (@njit(nogil=True))**

multiprocessing в python

```
from multiprocessing import Process

def heavy_task(args) -> None:
    """Абстрактная тяжёлая CPU-bound задача"""
    raise NotImplementedError

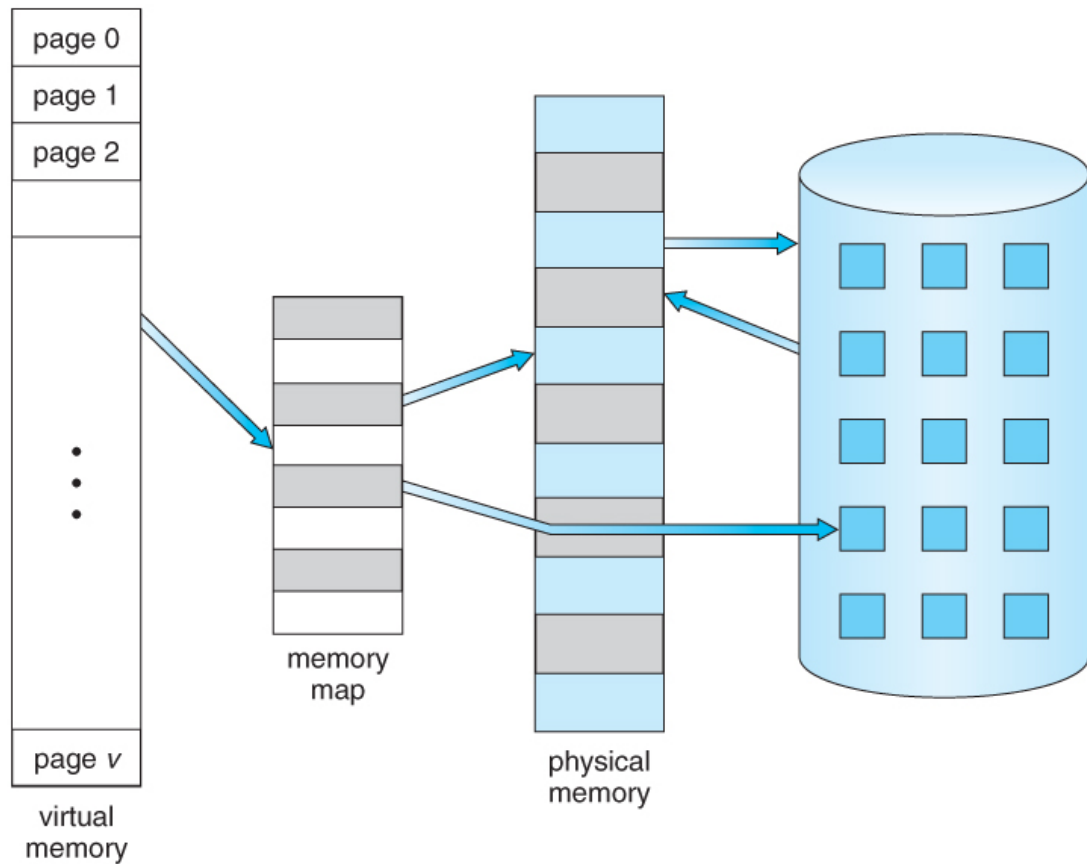
if __name__ == "__main__":
    args = [...]
    processes = []
    for args_ in args:
        p = Process(target=heavy_task, args=(args_,))
        p.start()
        processes.append(p)

    # ждём завершения всех процессов
    for p in processes:
        p.join()
```

Способы создания процессов

```
import multiprocessing  
multiprocessing.set_start_method('spawn', force=True)
```

OS pages



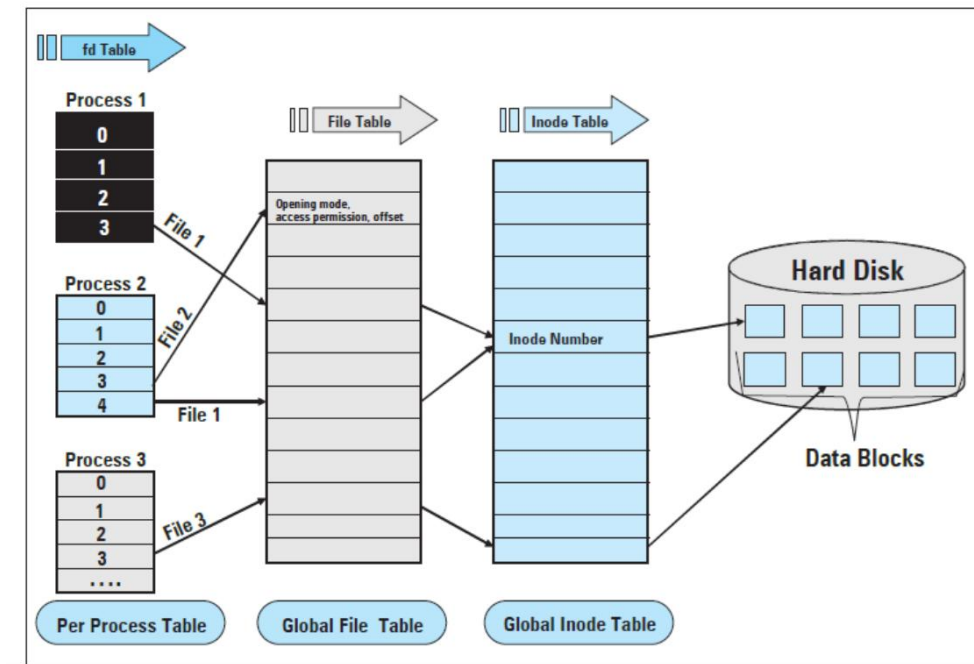
Файловый дескриптор (fd)

Это небольшое целое **число**, которое процесс использует как ссылку на открытый ресурс ядра операционной системы: файл, сокет, pipe и тд:

- Индекс в таблице файловых дескрипторов процесса
- По этому индексу процесс через ядро получает доступ к записи открытого файла.

Таблица Global File Table – таблица на уровне ядра ОС, где хранится состояние конкретного открытия файла (режим открытия, offset и тд)

Таблица Global Inode Table (index node) – описывает файлы как сущности файловой системы (тип, размер, права доступов и тд)



Например:

fd=0 – **stdin**, стандартный ввод
fd=1 – **stdout**, стандартный вывод
fd=2 – **stderr**, стандартный поток ошибок

Способы создания процессов. fork

fork — способ запуска процесса, при котором ОС **клонировает** текущий процесс:

- Дочерний процесс стартует как **копия** родителя: у него **своё** виртуальное адресное пространство с тем же содержимым, при этом его страницы изначально отображаются на те же физические страницы памяти (**copy-on-write**)
- Память реализована через **copy-on-write (COW)**: физические страницы могут быть общими, пока никто не пишет; при записи страница копируется
- Наследуются открытые файлы/сокеты
- Открытые файлы/сокеты/пайпы (**таблица файловых дескрипторов**) копируются, но указывают на те же объекты ядра

Плюсы:

- быстрый старт
- на момент старта много состояний уже готово (импортированные модули, объекты в памяти, состояние интерпретатора)

Минусы:

- риск проблем при наличии нескольких потоков (deadlock после fork)
- плохо совместим с CUDA/некоторыми библиотеками
- поскольку наследует те же сокеты, файлы, может приводить к неожиданным багам

Copy-On-Write

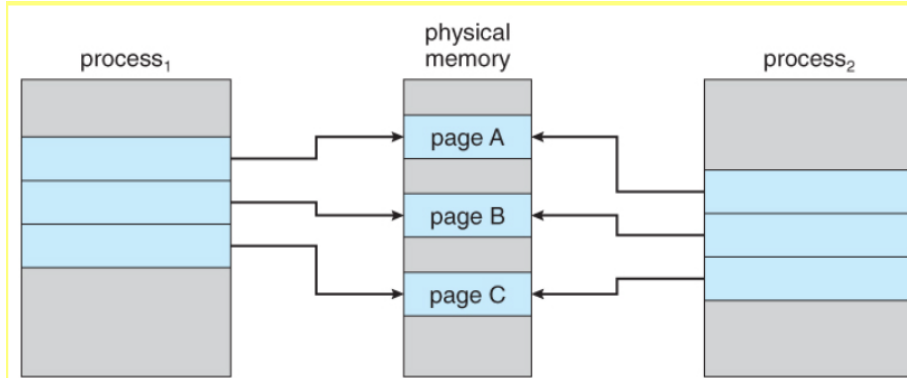


Figure 9.7 - Before process 1 modifies page C.

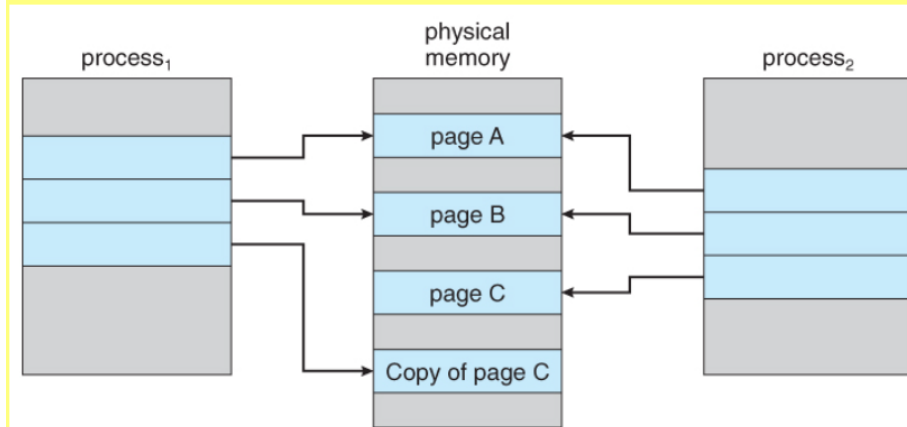


Figure 9.8 - After process 1 modifies page C.

Способы создания процессов. `spawn`

spawn — способ запуска процесса, при котором:

- создаётся **новый чистый** процесс:
- создаётся новое виртуальное адресное пространство: родительская память **не копируется**

В python:

- модули импортируются заново
- запускается новый экземпляр интерпретатора
- поскольку память не общая, python-объекты надо сериализовать – обычно через **pickle**

Плюсы:

- более предсказуемый поведение со старта
- безопаснее с несколькими потоками и асинхронным кодом
- обычно предпочтительнее для CUDA

Минусы:

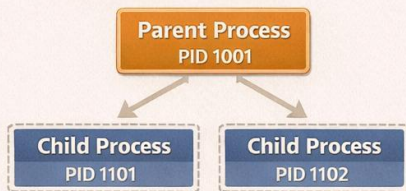
- медленнее старт
- требуется unpickle объекты

Способы создания процессов. fork vs spawn

Fork vs Spawn: два способа создания процесса

Fork

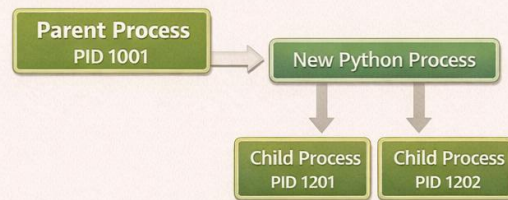
- Клонировать текущий процесс
 - Копия родителя
 - Наследование ресурсов



- ⚡ Быстрое и всё наследуется
- ⚠ Может быть нестабильно с потоками и CUDA

Spawn

- Запустить новый чистый процесс
 - Новый процесс Python
 - Чистая среда



- ✓ Безопасно и стабильно
- ⌚ Медленнее и требует пиклинг

Синхронизация процессов

- Те же концепции **Lock/RLock/semaphore**, что и случае многопоточности, но синхронизация в multiprocessing нужна либо для общего состояния (shared resources), либо встроено для **message passing** (Queue/Pipe), чтобы коммуникация была безопасной

IPC (Inter-Process Communication)

1. **Shared memory (общая память)** – процессы читают/пишут в один общий буфер (массив байтов) в RAM:

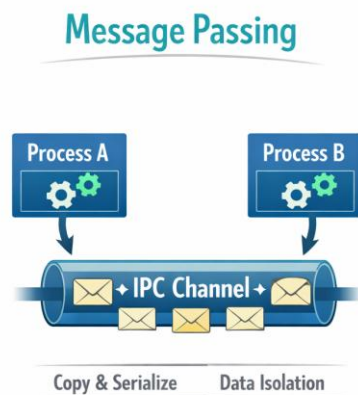
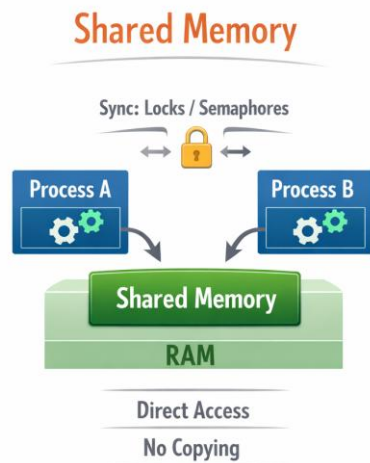
- минимум копирования
- но нужна синхронизация (locks/semaphores)

Например:

- **multiprocessing.shared_memory** – предоставляет общий массив байтов
- **multiprocessing.Array/Value** – типизированные shared-объекты (простые структуры), быстрый доступ

1. **Message passing (обмен сообщениями):**
процессы пересылают данные через IPC-канал (**Queue, Pipe**):

- копирование/сериализация
- зато проще изолировать процессы



IPC. Message passing

Pipe

Pipe – это IPC-канал (объект/механизм ОС) между процессами: **поток (stream) байтов** с буфером в **ядре**

При создании pipe ОС возвращает **два файловых дескриптора**, которые ссылаются на два конца одного и того же канала.

Классический pipe **однонаправленный**:

- **write end** — дескриптор, через который можно только писать
- **read end** — дескриптор, через который можно только читать



Queue

Queue (**multiprocessing.Queue**) – **FIFO**, но в multiprocessing это прежде всего канал обмена объектами между процессами, удобный для схемы **producer-consumer**.

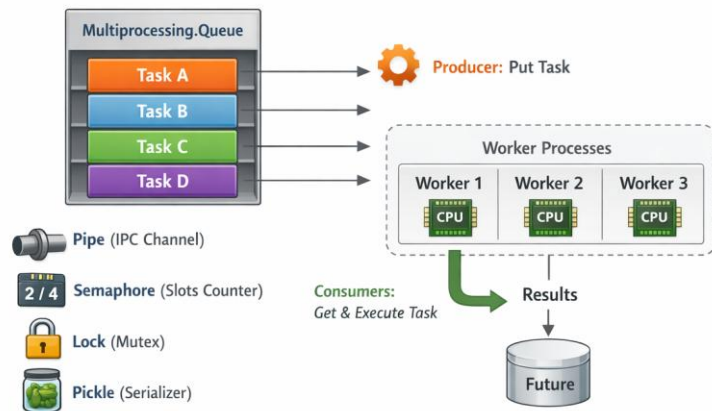
Состоит из:

- **Pipe** – канал для байтов между процессами
- **pickle** – сериализация Python-объектов
- **Semaphore** – ограничивает количество задач/запросов по **maxsize** – **backpressure**
- **Lock (mutex)** – защита критических секций записи

Схема *producer - consumer*:

1. Producer-процесс кладёт задачу в очередь:
queue.put(task)
2. Consumer-процесс забирает: **task = queue.get()**
3. Consumer выполняет задачу
4. Если задач больше нет – consumer блокируется на **get()** и ожидает
5. Когда producer кладёт новую задачу – consumer просыпается и продолжает

```
from multiprocessing import Queue  
queue = Queue(maxsize=3)
```



Manager

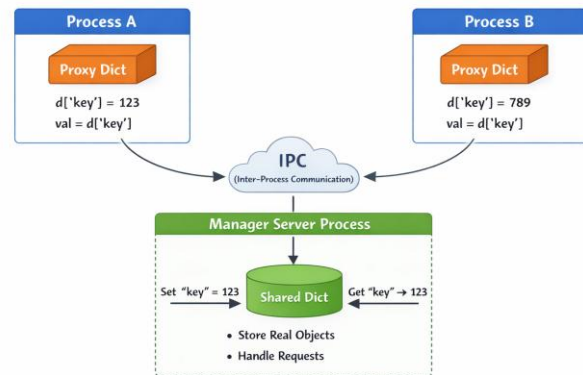
multiprocessing.Manager() запускает **отдельный процесс-сервер** (manager server process), который хранит **реальные объекты** (например, **dict**, **list**, **Queue**)

Процессы-«клиенты» получают не сам объект, а **прокси (proxy)** — объект-заместитель, который:

- выглядит как обычный **dict/list/Queue** (тот же интерфейс),
- но каждую операцию (**put/get**, **d[key]=...**, **append**, ...) отправляет в manager-процесс по IPC,
- manager выполняет операцию над реальным объектом и возвращает результат

Связь с fork/spawn

- При **fork** многие IPC-ресурсы (в т.ч. **mp.Queue**) удобно наследуются детьми
- При **spawn** аргументы должны быть serializable/picklable; прокси от **Manager** часто проще передавать, чем сам IPC-объект.



ProcessPoolExecutor

- Высокоуровневая абстракция над multiprocessing: пул рабочих процессов
- Подходит для “выполнить много независимых задач (**CPU-bound**) – получить результаты”
- Накладные расходы: создание процессов + pickle/сериализация данных

Что внутри:

- Создает очередь для управления задачами
- Передаёт данные через pickle → task (функция + аргументы должны быть picklable) – главный процесс при создании пула процессов передает task созданным процессам
- Создает объект **future**, служащий контейнером для результата
- Реализует корректное завершение процессов из пула (потокобезопасные **shutdown** и **submit**)

```
from concurrent.futures import ProcessPoolExecutor, as_completed

args_list = [...]
results = []

with ProcessPoolExecutor(max_workers=max_workers) as pool:
    futures = [pool.submit(func, *args) for args in args_list]

    for future in as_completed(futures):
        results.append(future.result())
```

Пример. cv (наивная версия)

```
def parallel_training(dataset, info, savepath, cv=5, gpu_ids=[0, 1, 2, 3], random_state=0, config=None):
    num_gpus = len(gpu_ids)
    kf = KFold(n_splits=cv, shuffle=True, random_state=random_state)
    processes = []

    for fold, (train_idx, val_idx) in enumerate(kf.split(dataset)):
        gpu_idx = fold % num_gpus
        device_id = gpu_ids[gpu_idx]
        p = multiprocessing.Process(
            target=train,
            args=(dataset, info, train_idx, val_idx, gpu_idx, device_id, fold, savepath, config)
        )
        p.start()
        processes.append(p)

    # Wait for all processes to finish
    for p in processes:
        p.join()

    return processes
```

Пример. cv

```
mp.set_start_method('spawn', force=True)

def parallel_training(dataset, info, savepath, cv=5,
                      gpu_ids=[0, 1, 2], random_state=0,
                      config=None):
    kf = KFold(n_splits=cv, shuffle=True, random_state=random_state)

    # Manager нужен, чтобы очередь GPU была проху
    # и её можно было передать в submit()
    with mp.Manager() as manager:
        gpu_queue = manager.Queue()
        for gpu_id in gpu_ids:
            gpu_queue.put(gpu_id)

    futures = []
    with ProcessPoolExecutor(max_workers=cv) as executor:
        iterator = kf.split(dataset)
        for fold, (train_idx, val_idx) in enumerate(iterator):
            futures.append(executor.submit(
                run_fold_with_gpu_queue,
                dataset, info, train_idx, val_idx, fold, savepath, config,
                gpu_queue
            ))

    # as_completed — получать результаты по мере завершения фолдов
    results = [f.result() for f in as_completed(futures)]
    return results
```

```
def run_fold_with_gpu_queue(
    dataset, info, train_idx,
    val_idx, fold, savepath, config, gpu_q
):
    # 1) занять GPU (если свободных нет — блокируемся)
    gpu_id = gpu_q.get()
    try:
        os.environ["CUDA_VISIBLE_DEVICES"] = str(gpu_id)

        train(dataset, info, train_idx, val_idx,
              gpu_idx=0, device_id=gpu_id,
              fold=fold, savepath=savepath, config=config)

        return fold, gpu_id

    except Exception as e:
        print('Exception in process', e)
    finally:
        gpu_q.put(gpu_id)
```

ВОТ И ДУМАЙТЕ...